Boolean Algebra

Discrete Math, Fall 2025

Konstantin Chukharev

Boolean Algebra

“Мы почитаем всех нулями,
А единицами — себя.”

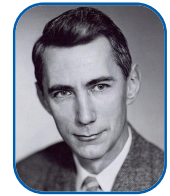
— А.С. Пушкин, «Евгений Онегин»

Gottfried Wilhelm Leibniz

George Boole

Augustus De Morgan

Charles Sanders Peirce



Claude Shannon

From Algebra to Digital Circuits

Historical note: George Boole's groundbreaking work *The Laws of Thought* (1854) created an algebraic system for logic a full century before the first electronic computers. He showed that logical reasoning could be captured by equations, replacing Aristotle's verbal arguments with mathematical symbols. This insight became the theoretical foundation for the digital age.

Key idea: Just as ordinary algebra works with numbers and operations like $+$ and $\times$, Boolean algebra works with **truth values** (0 and 1, or `false` and `true`) and operations like AND, OR, and NOT.

Historical milestones:

- **1703:** Leibniz describes binary arithmetic
- **1854:** Boole publishes algebraic logic
- **1937:** Shannon applies it to circuits
- **1940s:** First digital computers

Modern applications:

- Processor design (CPUs, GPUs)
- Formal verification (correctness proofs)
- Database optimization (queries)
- AI reasoning (SAT, planning)

Boolean Values: 0 and 1

In Boolean algebra, we work with exactly two values:

- **0** (`false`, off, low voltage)
- **1** (`true`, on, high voltage)

Example: In different contexts:

- **Mathematics:** "Is $x > 5$?" $\longrightarrow$ `true` (1) or `false` (0)
- **Programming:** `if (x && y)` $\longrightarrow$ evaluates to `true` or `false`
- **Circuits:** Wire voltage $\longrightarrow$ high (1) or low (0)
- **Sets:** Is element in set? $\longrightarrow$ yes (1) or no (0)

Why this matters: Modern computers represent all information using binary (0s and 1s).

Boolean algebra is the mathematics that makes digital computation possible.

What You Will Learn

In this lecture, you'll master Boolean algebra from foundations to applications:

1. Build Boolean expressions and evaluate them using **truth tables**
2. Prove identities using the **fundamental laws** (commutativity, De Morgan's laws, distributivity)
3. **Synthesize any Boolean function** from specifications using CNF, DNF, and Karnaugh maps
4. **Simplify complex expressions** through algebraic manipulation and graphical methods
5. Determine which operation sets are **functionally complete** using Post's criterion
6. **Design real digital circuits** with logic gates, flip-flops, and stateful elements
7. Apply Boolean algebra to **modern CS problems**: binary decision diagrams, SAT solving, and formal verification

Our goal: You'll gain both theoretical understanding (algebraic structure, formal proofs) and practical skills (circuit design, expression minimization). By the end, you'll see how the same mathematical framework powers everything from smartphone processors to AI planning algorithms.

Variables and Expressions

The Language of Boolean Logic

Just like natural languages have words and grammar rules, Boolean algebra has its own building blocks and composition rules.

Natural Language:

- Words: “cat”, “dog”
- Connectors: “and”, “or”, “not”
- Sentences: “Cat is black and dog is not white”

Boolean Algebra:

- Variables: x, y
- Operations: $\wedge$, $\vee$, $\neg$
- Expressions: $x \wedge \neg y$

Goal: Learn the “grammar” of Boolean logic — how to write and read Boolean expressions that represent logical relationships.

Boolean Variables: The Atoms

Definition 1: A *Boolean variable* is a variable that can take only one of two values: 0 (false) or 1 (true).

Think of Boolean variables as yes/no questions:

Example: Real-world Boolean variables (in programming):

- `is_logged_in` — Is the user logged in? (yes/no)
- `has_permission` — Does user have permission? (yes/no)
- `sensor_triggered` — Did the sensor activate? (yes/no)
- $x > 5$ — Is x greater than 5? (true/false)

Why binary? Digital circuits use voltage levels (high/low), making binary the natural choice for computation. Every piece of data in your computer is ultimately represented as 0s and 1s.

Boolean Operations: Connecting Ideas

We combine Boolean variables using operations (connectives) to express complex logic:

Definition 2: Basic Boolean operations:

- **NOT** ($\neg x$ or $\overline{x}$) — *reverses* the value
- **AND** ($x \wedge y$ or $x \cdot y$) — true only if *both* are true
- **OR** ($x \vee y$ or $x + y$) — true if *at least one* is true

Example: Access control logic:

- `is_admin OR has_permission` — User needs admin status OR explicit permission
- `is_logged_in AND NOT is_banned` — User must be logged in AND not banned
- `NOT (sensor1 AND sensor2)` — Not both sensors triggered simultaneously

Watch out: "OR" in Boolean logic is *inclusive* (can be both), unlike everyday speech where "coffee or tea" usually means "pick one."

Derived Operations: Shortcuts

More complex operations built from basic ones:

Definition 3: Derived Boolean operations:

- **XOR** ($x \oplus y$): true if *exactly one* is true
  - Formula: $(x \wedge \neg y) \vee (\neg x \wedge y)$
  - Example: "Dessert or coffee" (pick one, not both)
- **Implication** ($x \rightarrow y$): "if x then y"
  - Formula: $\neg x \vee y$
  - Example: "If it rains, bring umbrella"
- **Equivalence** ($x \Longleftrightarrow y$): true when values *match*
  - Formula: $(x \rightarrow y) \wedge (y \rightarrow x)$
  - Example: "Light is on if and only if switch is up"

Building Boolean Expressions

Definition 4: A *Boolean expression* is built recursively from variables and operations:

1. Variables (x, y, z) and constants ($0, 1$) are expressions
2. If f and g are expressions, so are: $\neg f$, $f \wedge g$, $f \vee g$, $f \oplus g$, $f \to g$, $f \iff g$

Example: Progressive complexity:

1. x — atomic variable
2. $\neg x$ — negation
3. $x \wedge y$ — two variables connected
4. $(x \wedge y) \vee z$ — nested expression
5. $\neg(x \vee (y \wedge z))$ — deeply nested
6. $(x \to y) \wedge (y \to z)$ — transitivity pattern

Key idea: Operations work on *any* expressions, not just variables. This compositionality lets us build arbitrarily complex formulas.

Truth Tables

Definition 5: A *truth table* is a complete list of all possible input combinations for a Boolean expression and their corresponding output values.

Note: For n Boolean variables, there are 2^n possible input combinations.

Example: Truth table for AND operation ($x \wedge y$):

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

Truth Tables for Compound Expressions

We can build truth tables for complex expressions step by step.

Example: Truth table for $(x \wedge y) \vee (\neg x \wedge z)$:

x	y	z	$x \wedge y$	$\neg x \wedge z$	$(x \wedge y) \vee (\neg x \wedge z)$
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	0	0
0	1	1	0	1	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	1	0	1
1	1	1	1	0	1

Method: Build intermediate columns for subexpressions, then combine them for the final result.

Tautologies, Contradictions, Contingencies

Definition 6:

- A *tautology* is a Boolean expression that is always true (all 1s in output column)
- A *contradiction* is a Boolean expression that is always false (all 0s in output column)
- A *contingency* is a Boolean expression that is sometimes true and sometimes false

Example:

- **Tautology:** $x \vee \neg x$ (law of excluded middle)
- **Contradiction:** $x \wedge \neg x$ (law of non-contradiction)
- **Contingency:** $x \wedge y$ (depends on values of x and y)

Warning: In Boolean algebra, we typically care about expressions that are contingencies—those that represent actual functions. Tautologies and contradictions are constant functions.

Logical Equivalence

Definition 7: Two Boolean expressions f and g are *logically equivalent* (written $f \equiv g$) if they have identical truth tables—they produce the same output for every possible input combination.

Example: Show that $\neg(x \wedge y) \equiv \neg x \vee \neg y$ (De Morgan's law):

x	y	$\neg(x \wedge y)$	$\neg x \vee \neg y$	**Equal?**
0	0	1	1	✓
0	1	1	1	✓
1	0	1	1	✓
1	1	0	0	✓

Since all rows match, $\neg(x \wedge y) \equiv \neg x \vee \neg y$. ■

Evaluating Boolean Expressions

Given values for all variables, we can evaluate any Boolean expression.

Example: Evaluate $f(x, y, z) = (x \vee y) \wedge (\neg x \vee z)$ when $x = 1, y = 0, z = 1$:

$$\begin{aligned} f(1, 0, 1) &= (1 \vee 0) \wedge (\neg 1 \vee 1) \\ &= 1 \wedge (0 \vee 1) \\ &= 1 \wedge 1 \\ &= 1 \end{aligned}$$

Practice skill: Being able to quickly evaluate Boolean expressions is essential for debugging circuits and verifying logical designs.

Simplifying Boolean Expressions

Simple algebraic manipulation can simplify expressions:

Example: Simplify $f(x, y) = (x \wedge y) \vee (x \wedge \neg y)$:

$$\begin{aligned} f(x, y) &= (x \wedge y) \vee (x \wedge \neg y) \\ &= x \wedge (y \vee \neg y) && \text{(distributivity)} \\ &= x \wedge 1 && \text{(excluded middle)} \\ &= x && \text{(identity)} \end{aligned}$$

Example: Simplify $g(x, y, z) = (x \vee y) \wedge (x \vee \neg y)$:

$$\begin{aligned} g(x, y, z) &= (x \vee y) \wedge (x \vee \neg y) \\ &= x \vee (y \wedge \neg y) && \text{(distributivity)} \\ &= x \vee 0 && \text{(contradiction)} \\ &= x && \text{(identity)} \end{aligned}$$

Boolean Algebra Structure

From Expressions to Algebraic Structure

We have seen Boolean expressions as formulas. Now we study their *algebraic structure*: the operations and laws that govern how they behave.

Why formalize? Just as group theory abstracts the common structure of numbers, symmetries, and transformations, Boolean algebra abstracts the structure of logic, sets, and circuits.

What we already have:

- Variables and constants
- Operations: $\neg$, $\wedge$, $\vee$
- Expressions
- Truth tables

What we'll add:

- Formal axioms
- Fundamental laws
- Proof techniques
- Connection to lattices

Boolean Algebra: Formal Definition

Definition 8: A *Boolean algebra* is a set B with:

- Two binary operations: $\vee$ (join) and $\wedge$ (meet)
- One unary operation: $\neg$ (complement)
- Two special elements: 0 (bottom) and 1 (top)

satisfying the axioms on the next slide.

Example: The two-element Boolean algebra $\{0, 1\}$ with:

- $0 \vee 0 = 0, 0 \vee 1 = 1, 1 \vee 0 = 1, 1 \vee 1 = 1$
- $0 \wedge 0 = 0, 0 \wedge 1 = 0, 1 \wedge 0 = 0, 1 \wedge 1 = 1$
- $\neg 0 = 1, \neg 1 = 0$

Boolean Algebra Axioms

Definition 9 (Axioms of Boolean Algebra): For all $x, y, z \in B$:

1. **Commutativity:** $x \vee y = y \vee x$ and $x \wedge y = y \wedge x$
2. **Associativity:** $(x \vee y) \vee z = x \vee (y \vee z)$ and $(x \wedge y) \wedge z = x \wedge (y \wedge z)$
3. **Distributivity:** $x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$ and $x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$
4. **Identity:** $x \vee 0 = x$ and $x \wedge 1 = x$
5. **Complement:** $x \vee \neg x = 1$ and $x \wedge \neg x = 0$

Note: Both $\wedge$ and $\vee$ distribute over each other — this is special to Boolean algebra (unlike ordinary arithmetic where only $\times$ distributes over $+$).

Examples of Boolean Algebras

Example (Power Set): For any set A, $(\mathcal{P}(A), \cup, \cap, \neg, \emptyset, A)$ is a Boolean algebra:

- Join: $\cup$ (union)
- Meet: $\cap$ (intersection)
- Complement: $X^c = A \setminus X$
- Bottom: $\emptyset$, Top: A

Example (Binary Strings): For n-bit binary strings with bitwise operations:

- Join: bitwise OR
- Meet: bitwise AND
- Complement: bitwise NOT
- Bottom: 000…0, Top: 111…1

Key insight: The same algebraic structure appears in logic, set theory, circuits, and more.

Fundamental Laws (Derived Properties)

From the axioms, we can prove many useful identities:

Theorem 1 (Basic Laws): For all $x, y \in B$:

1. **Idempotence:** $x \vee x = x$ and $x \wedge x = x$
2. **Absorption:** $x \vee (x \wedge y) = x$ and $x \wedge (x \vee y) = x$
3. **Null (Domination):** $x \vee 1 = 1$ and $x \wedge 0 = 0$
4. **Double Negation:** $\neg\neg x = x$
5. **Complement of Constants:** $\neg 0 = 1$ and $\neg 1 = 0$

Note: These can all be proven from the axioms using algebraic manipulation.

Proving Identities: Idempotence

Theorem 2: $x \vee x = x$ for all x in a Boolean algebra.

Proof:

$$
\begin{aligned}
x \vee x &= (x \vee x) \wedge 1 && \text{(identity)} \\
&= (x \vee x) \wedge (x \vee \neg x) && \text{(complement)} \\
&= x \vee (x \wedge \neg x) && \text{(distributivity)} \\
&= x \vee 0 && \text{(complement)} \\
&= x && \text{(identity)}
\end{aligned}
$$

$\square$

Method: Use axioms strategically — introduce 1 or 0, apply distributivity, then simplify.

Proving Identities: Absorption

Theorem 3: $x \vee (x \wedge y) = x$ for all x, y in a Boolean algebra.

Proof:

$$\begin{aligned} x \vee (x \wedge y) &= (x \wedge 1) \vee (x \wedge y) && \text{(identity)} \\ &= x \wedge (1 \vee y) && \text{(distributivity)} \\ &= x \wedge 1 && \text{(null law)} \\ &= x && \text{(identity)} \end{aligned}$$

$\square$

Example: Similarly, $x \wedge (x \vee y) = x$ by dual reasoning.

The Duality Principle

Theorem 4 (Duality Principle): If an identity holds in any Boolean algebra, then the *dual* identity (obtained by swapping $\vee \Leftrightarrow \wedge$ and $0 \Leftrightarrow 1$) also holds.

Example:

- Identity law: $x \vee 0 = x$, and its dual: $x \wedge 1 = x$
- Absorption: $x \vee (x \wedge y) = x$, and its dual: $x \wedge (x \vee y) = x$
- De Morgan: $\neg(x \vee y) = \neg x \wedge \neg y$, and its dual: $\neg(x \wedge y) = \neg x \vee \neg y$

Power of duality: Every theorem you prove gives you *two* theorems for free!

De Morgan's Laws

Theorem 5 (De Morgan's Laws): For all x, y in a Boolean algebra:

- $\neg(x \vee y) = \neg x \wedge \neg y$
- $\neg(x \wedge y) = \neg x \vee \neg y$

Proof: We prove $\neg(x \vee y) = \neg x \wedge \neg y$ by showing that $\neg x \wedge \neg y$ is the complement of $x \vee y$.

Need to verify:

1. $(x \vee y) \vee (\neg x \wedge \neg y) = 1$
2. $(x \vee y) \wedge (\neg x \wedge \neg y) = 0$

(Full proof left as exercise — use distributivity and complement laws.) $\square$

Common application: Negating complex conditions: `NOT (is_admin OR has_permission)` becomes `NOT is_admin AND NOT has_permission`.

Connection to Lattices

Definition 10: A *lattice* is a partially ordered set where any two elements x, y have:

- A *least upper bound* (join): $x \vee y$
- A *greatest lower bound* (meet): $x \wedge y$

Definition 11: A *bounded distributive lattice* is a lattice with bottom $\bot$, top $\top$, and distributivity:

$$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$$

Definition 12: A *complemented lattice* is a bounded lattice where every element x has a complement y such that $x \vee y = \top$ and $x \wedge y = \bot$.

Theorem 6: Every Boolean algebra is a complemented bounded distributive lattice.

Complement Uniqueness

Theorem 7: In a Boolean algebra, each element has a *unique* complement.

Proof: Suppose y and z are both complements of x. Then:

$$
\begin{aligned}
y &= y \wedge 1 && \text{(identity)} \\
&= y \wedge (x \vee z) && (z \text{ is complement of } x) \\
&= (y \wedge x) \vee (y \wedge z) && \text{(distributivity)} \\
&= 0 \vee (y \wedge z) && (y \text{ is complement of } x) \\
&= (x \wedge z) \vee (y \wedge z) && (z \text{ is complement of } x) \\
&= (x \vee y) \wedge z && \text{(distributivity)} \\
&= 1 \wedge z && (y \text{ is complement of } x) \\
&= z && \text{(identity)}
\end{aligned}
$$

Therefore $y = z$. $\square$

Summary of Boolean Algebra Structure

What we've established:

1. Boolean algebra is an algebraic structure with operations $\vee$, $\wedge$, $\neg$ and elements 0, 1
2. Five fundamental axioms govern the structure
3. Many useful laws (idempotence, absorption, De Morgan's) follow from the axioms
4. Duality principle: swap $\vee \Leftrightarrow \wedge$ and $0 \Leftrightarrow 1$ to get new theorems
5. Boolean algebras are complemented bounded distributive lattices
6. Complements are unique

Next: We'll use this algebraic structure to develop normal forms and minimization techniques.

Normal Forms

From Structure to Representation

We've established the algebraic structure of Boolean algebra.

Now: how do we *represent* any Boolean function systematically?

Goal: Find standard forms that can express *any* Boolean function. This lets us:

1. Synthesize circuits from specifications (truth tables)
2. Compare functions for equivalence
3. Systematically simplify expressions

What we have:

- Boolean algebra axioms
- Proof techniques
- Specific functions

What we need:

- Standard representations
- Construction algorithms
- Completeness guarantees

Building Blocks: Literals

Definition 13: A *literal* is either a Boolean variable or its negation.

Think of literals as the simplest meaningful statements:

Example: For variables x, y, z:

- **Positive literals:** x, y, z — "is true"
- **Negative literals:** $\neg x$, $\neg y$, $\neg z$ — "is false"

Example (Real-world interpretation):

- x might mean "user is logged in"
- $\neg x$ means "user is NOT logged in"
- The expression $(x \wedge \neg y) \vee z$ combines three literals: x, $\neg y$, and z

Key idea: Literals are the atoms of Boolean logic. Every complex expression is ultimately built from these simple building blocks using $\wedge$ and $\vee$.

Terms and Clauses

Definition 14:
- A *term* (or *product*) is a conjunction (AND) of literals
- A *clause* (or *sum*) is a disjunction (OR) of literals

Terms (Products):
- $x \wedge y$
- $x \wedge \neg y \wedge z$
- $\neg x \wedge \neg y \wedge \neg z$

"ALL of these must be true"

Clauses (Sums):
- $x \vee y$
- $x \vee \neg y \vee z$
- $\neg x \vee \neg y \vee \neg z$

"At least ONE must be true"

Example:
- Term $x \wedge \neg y \wedge z$ is true only when: $x = 1$, $y = 0$, $z = 1$
- Clause $x \vee \neg y \vee z$ is true when: $x = 1$ OR $y = 0$ OR $z = 1$ (or any combination)

Note: A single literal is both a 1-term and a 1-clause. Constants: 0 (empty term) and 1 (empty clause).

Disjunctive Normal Form (DNF)

Definition 15 (DNF): A Boolean formula is in *Disjunctive Normal Form (DNF)* if it is a disjunction (OR) of terms.

General form: $(t_1) \vee (t_2) \vee \ldots \vee (t_k)$ where each t_i is a term (conjunction of literals).

Example: DNF formulas:

- $(x \wedge y) \vee (\neg x \wedge z)$ — "(x and y) OR ($\neg x$ and z)"
- $(x \wedge \neg y \wedge z) \vee (\neg x \wedge y) \vee z$ — three alternatives
- $x \vee (y \wedge \neg z)$ — still DNF (mixed form)

Example: **NOT** in DNF:

- $(x \vee y) \wedge z$ — this is CNF (AND of ORs)
- $x \wedge (y \vee z)$ — OR nested inside AND violates DNF structure

Disjunctive Normal Form (DNF) [2]

Intuition: DNF says "the output is 1 if ANY of these scenarios happen," where each scenario is a specific combination of variable values.

Conjunctive Normal Form (CNF)

Definition 16 (CNF): A Boolean formula is in *Conjunctive Normal Form (CNF)* if it is a conjunction (AND) of clauses.

General form: $(c_1) \wedge (c_2) \wedge \ldots \wedge (c_k)$ where each c_i is a clause (disjunction of literals).

Example: CNF formulas:

- $(x \vee y) \wedge (\neg x \vee z)$ — both constraints must hold
- $(x \vee \neg y \vee z) \wedge (\neg x \vee y) \wedge z$ — three constraints
- $x \wedge (y \vee \neg z)$ — still CNF (mixed form)

Example: **NOT** in CNF:

- $(x \wedge y) \vee z$ — this is DNF (OR of ANDs)
- $x \vee (y \wedge z)$ — AND nested inside OR violates CNF structure

CNF vs DNF

Intuition: CNF says "ALL of these constraints must be satisfied," where each constraint offers multiple ways to be true.

DNF says "the output is 1 if ANY of these scenarios happen," where each scenario is a specific combination of variable values.

Duality: DNF and CNF are *dual* forms. Swap $\wedge$ and $\vee$ to convert between them.

Minterms and Maxterms

Definition 17: For n variables:

- A *minterm* is a term containing all n variables (each exactly once, positive or negated)
- A *maxterm* is a clause containing all n variables (each exactly once, positive or negated)

Example: For variables x, y, z:

- **Minterms:** $(x \wedge y \wedge z)$, $(x \wedge y \wedge \neg z)$, $(x \wedge \neg y \wedge z)$, ...
- **Maxterms:** $(x \vee y \vee z)$, $(x \vee y \vee \neg z)$, $(x \vee \neg y \vee z)$, ...

There are exactly $2^n = 2^3 = 8$ minterms and 8 maxterms for 3 variables.

Key property:

- Each minterm is 1 for **exactly ONE** input combination
- Each maxterm is 0 for **exactly ONE** input combination
- This makes them perfect building blocks for representing any function!

Minterm and Maxterm Indexing

We can index minterms and maxterms by their binary representations:

Example: For x, y, z (interpreting as bits: xyz):

Index	**Binary**	**Minterm m_i**	**Maxterm M_i**
0	000	$\neg x \land \neg y \land \neg z$	$x \lor y \lor z$
1	001	$\neg x \land \neg y \land z$	$x \lor y \lor \neg z$
2	010	$\neg x \land y \land \neg z$	$x \lor \neg y \lor z$
3	011	$\neg x \land y \land z$	$x \lor \neg y \lor \neg z$
4	100	$x \land \neg y \land \neg z$	$\neg x \lor y \lor z$
5	101	$x \land \neg y \land z$	$\neg x \lor y \lor \neg z$
6	110	$x \land y \land \neg z$	$\neg x \lor \neg y \lor z$
7	111	$x \land y \land z$	$\neg x \lor \neg y \lor \neg z$

Sum of Products (SoP)

Definition 18: A *sum of products (SoP)* (also called *canonical sum of minterms*) is a DNF where each term is a minterm.

General form: $f(x_1, ..., x_n) = \bigvee_{i \in I} m_i$ where $I \subseteq \{0, 1, ..., 2^n - 1\}$

Example: For the function with truth table:

x	y	z	f
0	0	0	0
0	0	1	**1**
0	1	0	0
0	1	1	**1**
1	0	0	0
1	0	1	0
1	1	0	**1**
1	1	1	**1**

Sum of Products (SoP) [2]

Algorithm: Pick rows where $f = 1$ (rows 1, 3, 6, 7):

$$\begin{aligned} f &= m_1 \lor m_3 \lor m_6 \lor m_7 \\ &= (\neg x \land \neg y \land z) \lor (\neg x \land y \land z) \lor (x \land y \land \neg z) \lor (x \land y \land z) \end{aligned}$$

Recipe: To build SoP, take each 1 in the truth table and OR the corresponding minterms.

Product of Sums (PoS)

Definition 19: A *product of sums (PoS)* (also called *canonical product of maxterms*) is a CNF where each clause is a maxterm.

General form: $f(x_1, ..., x_n) = \bigwedge_{i \in I} M_i$ where $I \subseteq \{0, 1, ..., 2^n - 1\}$

Example: For the **same function**, use rows where $f = 0$ (rows 0, 2, 4, 5):

$$\begin{aligned} f &= M_0 \wedge M_2 \wedge M_4 \wedge M_5 \\ &= (x \vee y \vee z) \wedge (x \vee \neg y \vee z) \wedge (\neg x \vee y \vee z) \wedge (\neg x \vee y \vee \neg z) \end{aligned}$$

Two approaches:

- SoP uses 1s in truth table (where function is true)
- PoS uses 0s in truth table (where function is false)
- Both represent the same function!

Why PoS? Sometimes PoS is more compact than SoP (when function has fewer 0s than 1s).

Completeness of Normal Forms

Theorem 8: Every Boolean function can be represented in both CNF and DNF.

Proof *Sketch for DNF (SoP)*: Given a truth table:

1. For each row where output is 1, create the corresponding minterm
2. OR all these minterms together
3. The result is the function in DNF (SoP form)

Why this works:

- Each minterm is 1 for exactly one input combination
- ORing them gives 1 exactly when the function should be 1
- This construction is always possible and always correct

□

Power of synthesis: Any Boolean function can be built from its truth table!

Shannon Expansion

Theorem 9 (Shannon Expansion): For any Boolean function f and variable x:

$$f(x, y_1, ..., y_n) = (\neg x \wedge f(0, y_1, ..., y_n)) \vee (x \wedge f(1, y_1, ..., y_n))$$

$$f(x, y_1, ..., y_n) = (x \vee f(0, y_1, ..., y_n)) \wedge (\neg x \vee f(1, y_1, ..., y_n))$$

Example: Expand $f(x, y) = x \oplus y$ by x:

$$\begin{aligned} f(x, y) &= (\neg x \wedge f(0, y)) \vee (x \wedge f(1, y)) \\ &= (\neg x \wedge (0 \oplus y)) \vee (x \wedge (1 \oplus y)) \\ &= (\neg x \wedge y) \vee (x \wedge \neg y) \end{aligned}$$

This gives us the standard XOR representation!

Converting Between Forms

Converting between DNF and CNF can be tricky:

Example (DNF to CNF using De Morgan's laws): Start with DNF: $(x \wedge y) \vee (\neg x \wedge z)$

Step-by-step:

1. Negate: $\neg((x \wedge y) \vee (\neg x \wedge z))$
2. Apply De Morgan: $(\neg(x \wedge y)) \wedge (\neg(\neg x \wedge z))$
3. Apply De Morgan again: $(\neg x \vee \neg y) \wedge (x \vee \neg z)$
4. Double negate to get back: $\neg\neg((\neg x \vee \neg y) \wedge (x \vee \neg z))$

Result is CNF: $(\neg x \vee \neg y) \wedge (x \vee \neg z)$

Warning: Direct algebraic conversion can cause exponential blowup in formula size! For complex functions, use Karnaugh maps or other minimization techniques.

Alternative: Build CNF/DNF directly from truth table using the SoP/PoS methods.

Summary: Canonical Forms

What we've learned:

1. Literals, terms, clauses are the building blocks
2. DNF: OR of terms (conditions that make function true)
3. CNF: AND of clauses (constraints that must all hold)
4. Minterms/maxterms: complete terms/clauses with all variables
5. SoP/PoS: canonical forms using minterms/maxterms
6. Shannon expansion: recursive decomposition by variables
7. Every Boolean function has CNF and DNF representations

Next: Minimization techniques to reduce the size of these expressions (K-Maps, Quine-McCluskey).

Minimization

The Minimization Problem

We can synthesize any Boolean function from its truth table using SoP or PoS.

But the result is often **not minimal**.

Example: Consider $f(x, y, z) = (\neg x \wedge \neg y \wedge z) \vee (\neg x \wedge y \wedge z) \vee (x \wedge y \wedge \neg z) \vee (x \wedge y \wedge z)$

This CDNF has 4 terms with 3 literals each (12 literals total).

But we can simplify: $f = (\neg x \wedge z) \vee (x \wedge y)$ (only 4 literals!)

Why minimize?

- Fewer gates $\Rightarrow$ cheaper circuits
- Less power consumption
- Higher speed (fewer gate delays)
- Easier to understand and verify

The challenge:

Finding the minimal form is computationally hard (NP-complete in general).

We need practical techniques!

What Does "Minimal" Mean?

Different minimization criteria (goals) exist:

- **Minimum literals:** Fewest total literal occurrences
- **Minimum terms:** Fewest product terms (DNF) or clauses (CNF)
- **Minimum gates:** Fewest logic gates in circuit
- **Minimum levels:** Shortest signal propagation path (depth)

Example: Function: $f = AB + AC + BC$

- Has 6 literals, 3 terms
- Can be reduced to: $f = AB + AC$ (using consensus theorem)
- Now 4 literals, 2 terms

Usually, we minimize the number of literals (most common criterion).

Gray Code: Foundation of K-Maps

Definition 20: A *Gray code* is a binary encoding where consecutive values differ in exactly one bit.

Example: 3-bit Gray code sequence:

Decimal	Binary	Gray Code	Change
0	000	000	—
1	001	001	bit 0
2	010	011	bit 1
3	011	010	bit 0
4	100	110	bit 2
5	101	111	bit 0
6	110	101	bit 1
7	111	100	bit 0

Note: Here, bit 0 is LSB, bit 2 is MSB.

Converting Binary and Gray Code

Binary to Gray:
- Keep MSB (most significant bit)
- Each next bit: XOR current binary bit with previous binary bit

Gray to Binary:
- Keep MSB
- Each next bit: XOR current Gray bit with previous binary bit

Example: Binary $\texttt{1011}_2 \rightarrow$ Gray:
- Bit 3: 1 (keep MSB)
- Bit 2: $1 \oplus 0 = 1$
- Bit 1: $0 \oplus 1 = 1$
- Bit 0: $1 \oplus 1 = 0$
- **Result:** $\texttt{1110}_{\text{Gray}}$

Introduction to Karnaugh Maps

Definition 21: A *Karnaugh map (K-map)* is a 2D grid representation of a truth table, arranged using Gray code so that adjacent cells differ in exactly one variable.

Key insight: K-maps visualize the algebraic identity: $(x \wedge y) \vee (x \wedge \neg y) = x$

Adjacent 1s in the map represent terms that can be combined!

Advantages:

- Visual and intuitive
- Fast for small functions
- Shows all simplifications
- No complex computation

Limitations:

- Practical only for $\leq$ 5-6 variables
- Doesn't scale to large functions
- Requires manual grouping skill
- Can miss some patterns in 6+ vars

2-Variable K-Map: Step by Step

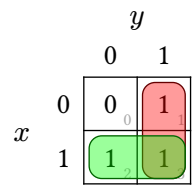
Example: Build K-map for $f(x, y) = x \vee y$.

Step 1: Create 2×2 grid and fill in the truth values

Step 2: Group adjacent 1s

- Horizontal pair (bottom row): $x = 1 \rightarrow$ gives term x
- Vertical pair (right column): $y = 1 \rightarrow$ gives term y

Result: $f = x \vee y$



3-Variable K-Map Structure

For 3 variables, we use a 4×2 grid (two variables for rows, one for columns):

xy \ z	0	1
00	0	1
01	2	3
11	6	7
10	4	5

Note: Row order: 00, 01, **11**, 10 (Gray code, NOT binary!)

This ensures top-bottom adjacency and wrap-around (torus structure).

Remember: The map wraps around — top and bottom rows are adjacent!

3-Variable K-Map: Complete Example

Example: Minimize $f(x, y, z) = \sum m(1, 3, 6, 7)$:

Step 1: Draw 4×2 grid and fill in the truth values

Step 2: Identify groupings

- Red group:
  - Variables: $x = 0$, $z = 1$, y varies
  - Term: $\neg x \wedge z$
- Green group:
  - Variables: $x = 1$, $y = 1$, z varies
  - Term: $x \wedge y$

Step 3: Write minimal DNF

$$f = (\neg x \wedge z) \vee (x \wedge y) = \overline{x}z + xy$$

xy \ z	0	1
00	0 (0)	1 (1)
01	0 (2)	1 (3)
11	1 (6)	1 (7)
10	0 (4)	0 (5)

How K-Map Grouping Works

Grouping principles:

- Variables that *change* within the group are eliminated
- Variables that stay *constant* remain in the term
- Larger groups $\Rightarrow$ more variables eliminated $\Rightarrow$ simpler terms

Example: Group of 2 cells in 3-var K-map:

Cell	xyz	**Analysis**
m_1	001	$x = 0, y = 0, z = 1$
m_3	011	$x = 0, y = 1, z = 1$
		$x = 0, z = 1, y$ varies

Result: $\neg x \wedge z$ (y eliminated)

4-Variable K-Map Structure

For 4 variables, use a 4×4 grid with Gray code on both axes:

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Important: Both rows and columns wrap around! Treat the map as a *torus*:

- Top $\Longleftrightarrow$ Bottom are adjacent
- Left $\Longleftrightarrow$ Right are adjacent
- Even corners can be grouped!

4-Variable K-Map: Complete Example

Example:

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Group	Cells	Pattern	Term
Corners	0, 2, 8, 10	$B = 0$, $D = 0$, A and C vary	$\overline{B}\,\overline{D}$
Vertical	5, 13	$B = 1$, $C = 0$, $D = 1$, A varies	$B\,\overline{C}\,D$

Minimal DNF: $f = \overline{B}\,\overline{D} + B\,\overline{C}\,D$

K-Map Grouping: Valid Group Sizes

Rules for valid groups:

1. Size must be a power of 2: 1, 2, 4, 8, or 16 cells
2. Shape must be rectangular (1×2, 2×2, 4×4, 1×8, *etc.*)
3. Can wrap around edges (torus topology)
4. Larger groups are always better (fewer literals)

Example: Relationship between group size and simplification:

Group Size	Variables Eliminated	Literals Left	Example Term
1 cell	0	4	$ABCD$
2 cells	1	3	ABC
4 cells	2	2	AB
8 cells	3	1	A
16 cells	4	0	1 (tautology)

K-Map Strategy: Optimal Grouping

Step-by-step grouping strategy:

1. Mark all 1s in the K-map
2. Find cells that can **only** be covered by one group (essential)
3. Group these with largest possible rectangles
4. Cover remaining 1s with largest possible groups
5. Minimize overlap (but overlap is allowed!)
6. Verify all 1s are covered

Pro tips:

- Start with isolated 1s (they need their own groups)
- Look for groups of 8, then 4, then 2
- Remember wraparound on all edges
- Corner cells can also form groups (2×2 block)

K-Maps with Don't-Care Conditions

Definition 22 (Don't-Care Conditions): Situations where output value doesn't matter (marked as $\times$):

- Invalid input combinations
- Outputs that are never used
- Incompletely specified functions

Example: BCD (Binary Coded Decimal) uses only 0-9:

Inputs 1010-1111 are don't-cares (invalid BCD)

Strategy with don't-cares:

- Treat $\times$ as 0 or 1 to maximize group sizes
- Include $\times$ in groups if it helps
- Don't create groups containing only $\times$ values

Don't-Care Example

Example: Function with don't-cares at positions 9, 11, 12, 15

AB \ CD	00	01	11	10
00	0 (0)	1 (1)	1 (3)	0 (2)
01	0 (4)	0 (5)	0 (7)	0 (6)
11	× (12)	1 (13)	× (15)	0 (14)
10	1 (8)	× (9)	× (11)	0 (10)

- Group 1: includes cells 1, 3 (filled with 1s) $\Rightarrow$ gives $\overline{A}\,\overline{B}\,D$
- Group 2: includes cells 8, 9, 12, 13 (uses don't-cares) $\Rightarrow$ gives $A\,\overline{C}$

Without don't-cares, we would need more terms to cover all 1s!

Algebraic Minimization

Beyond K-maps, we can minimize algebraically using Boolean laws:

Essential simplification laws:

Law	Sum Form	Product Form
Idempotent	$X + X = X$	$X \cdot X = X$
Absorption	$X + XY = X$	$X(X + Y) = X$
Combining	$XY + X\overline{Y} = X$	$(X + Y)\left(X + \overline{Y}\right) = X$
Consensus	$XY + \overline{X}Z + YZ$ $= XY + \overline{X}Z$	$(X + Y)\left(\overline{X} + Z\right)(Y + Z)$ $= (X + Y)\left(\overline{X} + Z\right)$

Step-by-Step Algebraic Minimization

Example: Minimize $f = ABC + AB\overline{C} + \overline{A}BC + \overline{A}\overline{B}C$:

Step 1: Look for combining opportunities

$$\begin{aligned} f &= ABC + AB\overline{C} + \overline{A}BC + \overline{A}\overline{B}C \\ &= AB\left(C + \overline{C}\right) + \overline{A}BC + \overline{A}\overline{B}C && \text{(factor)} \\ &= AB + \overline{A}BC + \overline{A}\overline{B}C && \text{(complement)} \end{aligned}$$

Step 2: Apply more combining

$$\begin{aligned} f &= AB + \overline{A}C\left(B + \overline{B}\right) && \text{(factor)} \\ &= AB + \overline{A}C && \text{(complement)} \end{aligned}$$

Final result: $f = AB + \overline{A}C$ (reduced from 12 to 4 literals!)

Consensus Theorem

Theorem 10 (Consensus Theorem): $XY + \overline{X}Z + YZ = XY + \overline{X}Z$

The term YZ is "absorbed" by the other two terms.

Proof:

$$\begin{aligned}
XY + \overline{X}Z + YZ &= XY + \overline{X}Z + \left(X + \overline{X}\right)YZ && \text{(complement)} \\
&= XY + \overline{X}Z + XYZ + \overline{X}YZ && \text{(distributive)} \\
&= XY(1 + Z) + \overline{X}Z(1 + Y) && \text{(factor)} \\
&= XY + \overline{X}Z && \text{(null: } 1 + X = 1\text{)}
\end{aligned}$$

□

Intuition: If YZ is true, then either XY or $\overline{X}Z$ must already be true, so YZ is redundant.

Multi-Level Minimization

Definition 23 (Multi-Level Logic): Instead of two-level SoP/PoS, use multiple levels of gates to share common subexpressions.

Example: Two-level: $f_1 = ABC + ABD$, $f_2 = ABC + ABE$

Total: 12 literals $(6 + 6)$

Multi-level with factoring:

- $T = AB$ (common factor)
- $f_1 = T(C + D)$
- $f_2 = T(C + E)$

Total: 6 literals + reuse of T

Trade-off: Multi-level uses fewer gates but has longer delay (more levels).

When to Use CNF vs DNF

DNF (Sum of Products):

- Few 1s in truth table
- OR-of-ANDs natural
- Most circuit designs
- Easy synthesis from minterms
- K-map method works well

CNF (Product of Sums):

- Few 0s in truth table
- AND-of-ORs structure
- SAT solver input
- Constraint representation
- Built from maxterms

Example: Consider two functions:

Truth Table	SoP Terms	PoS Clauses
2 ones, 6 zeros	2 terms	6 clauses
6 ones, 2 zeros	6 terms	2 clauses

Choose the form with fewer components!

Introduction to Quine-McCluskey

Definition 24: The *Quine-McCluskey algorithm* is a systematic tabular method for finding all prime implicants, guaranteed to produce a minimal form.

The Q-M algorithm has two phases:

1. Generate all prime implicants from minterms
2. Select a minimal set of prime implicants covering all minterms

When to use Q-M:

- More than 4-5 variables (K-maps impractical)
- Need guaranteed minimal solution
- Computer-aided design tools
- Can be automated (unlike K-maps)

Limitation: Complexity grows exponentially with variables — practical for ≤ 6-8 variables.

Quine-McCluskey Algorithm

The Q-M algorithm has two phases:

Phase 1: Generate Prime Implicants

1. List all minterms in binary
2. Group by number of 1s (Hamming weight)
3. Combine pairs differing in exactly one bit
4. Replace differing bit with dash (–)
5. Repeat until no more combinations
6. Uncombined terms are prime implicants

Phase 2: Select Minimal Cover

1. Build prime implicant chart
2. Find essential prime implicants
3. Use Petrick's method or heuristics for rest

Step-by-Step Example of Q-M Phase 1

Example: Minimize $f(A, B, C) = \sum m(1, 3, 5, 6, 7)$

Step 1: Group minterms by number of 1-bits (Hamming weight).

Group	Minterm	Binary
1	m_1	001
2	m_3	011
	m_5	101
	m_6	110
3	m_7	111

AB \ C	0	1
00	0 (0)	1 (1)
01	0 (2)	1 (3)
11	1 (6)	1 (7)
10	0 (4)	1 (5)

Q-M Phase 1: First Combination

Step 2: Try to combine each minterm in group i with each minterm in group $i+1$.

# of 1s	Minterm	Cube	Size 2 Implicants
1	m_1 ✓	001	$m_1 + m_3 = m_{1,3}$ ✓ 0–1
			$m_1 + m_5 = m_{1,5}$ ✓ –01
			$m_1 + m_6$ ✗ (differ in 3 positions)
2	m_3 ✓	011	$m_3 + m_7 = m_{3,7}$ ✓ –11
	m_5 ✓	101	$m_5 + m_7 = m_{5,7}$ ✓ 1–1
	m_6 ✓	110	$m_6 + m_7 = m_{6,7}$ ✓ 11–
3	m_7 ✓	111	—

Result: Five size-2 implicants formed: $m_{1,3}$, $m_{1,5}$, $m_{3,7}$, $m_{5,7}$, $m_{6,7}$

Note: Mark all "used" minterms with ✓ and all "unused" with ✗

Q-M Phase 1: Second Combination

Step 3: Try to combine each 2-size implicant with others having dashes in the SAME positions.

Dash pos.	Implicant	Pattern	Size 4 Implicant
pos. 0	$m_{1,5}$ ✓	–01	$m_{1,5} + m_{3,7} = m_{1,3,5,7}$ ✓ –-1
	$m_{3,7}$ ✓	–11	—
pos. 1	$m_{1,3}$ ✓	0–1	$m_{1,3} + m_{5,7} = m_{1,3,5,7}$ ✓ –-1
	$m_{5,7}$ ✓	1–1	—
pos. 2	$m_{6,7}$ ✗	11–	(no other implicant with dash at pos. 2)

Explanation:

- $m_{1,5}$ (–01) and $m_{3,7}$ (–11): dashes align at pos. 0, differ only in bit 1 ⇒ combine to $m_{1,3,5,7}$ (–-1) ✓
- $m_{1,3}$ (0–1) and $m_{5,7}$ (1–1): dashes align at pos. 1, differ only in bit 0 ⇒ combine to $m_{1,3,5,7}$ (–-1) ✓
- $m_{6,7}$ (11–): alone with dash at pos. 2 ⇒ cannot combine ✗

Result: One size-4 implicant formed: $m_{1,3,5,7}$ (–-1). One size-2 implicant remains uncombined: $m_{6,7}$ (11–).

Q-M Phase 1: Finding Prime Implicants

Step 4: Identify prime implicants (uncombined terms).

From Step 3:

- Size-4 implicant: $m_{1,3,5,7}$ ($--1$) is *prime* since there are no other size-4 implicants to combine with
- Size-2 implicants NOT marked with ✓: $m_{6,7}$ are *prime*

Pattern	Covers	Boolean Term
$--1$	$\{1, 3, 5, 7\}$	C
$11-$	$\{6, 7\}$	AB

These 4 prime implicants cannot be reduced further.

Prime Implicant Chart

Definition 25: A *prime implicant chart* is a table showing which minterms are covered by each prime implicant.

Note: Our goal in Q-M is to select a minimum set of prime implicants covering all minterms.

Example: For our example $\sum m(1, 3, 5, 6, 7)$:

Prime Implicant	m_1	m_3	m_5	m_6	m_7
C (−−1)	✓	✓	✓		✓
AB (11−)				✓	✓

Finding essential prime implicants:

- Look for columns with only ONE ✓ $\Rightarrow$ that PI is **essential**
  - Columns m_1, m_3, m_5 — only covered by $C \Rightarrow C$ is **essential**
  - Column m_6 — only covered by $AB \Rightarrow AB$ is **essential**

Prime Implicant Chart [2]

Selecting minimal cover:

- Must include: C (essential), covers $\{1, 3, 5, 7\}$
- Must include: AB (essential), covers $\{6, 7\}$

Minimal solution: $f = C + AB$ (3 literals)

Essential Prime Implicants

Definition 26: An *essential prime implicant* (EPI) is a prime implicant that covers at least one minterm not covered by any other prime implicant.

Example: Consider this prime implicant chart:

	m_i	m_j	m_k
PI_1	$\times$	$\times$	
PI_2		$\times$	$\times$
PI_3			$\times$

Key insight: If a minterm has *only one* $\times$, the corresponding prime implicant is *essential* and must be included in any minimal cover.

- Minterm m_i is covered only by $\mathrm{PI}_1 \Rightarrow \mathrm{PI}_1$ is *essential* $\Rightarrow$ it *must* be included
- Minterm m_j is covered by both PI_1 and $\mathrm{PI}_2 \Rightarrow$ no EPI
- Minterm m_k is covered by both PI_2 and $\mathrm{PI}_3 \Rightarrow$ no EPI

Conclusion: PI_1 *must* be included. For remaining minterms, choose either PI_2 or PI_3.

Petrick's Method

When multiple prime implicants remain after selecting essentials:

Definition 27: *Petrick's method* finds all combinations of prime implicants that cover the remaining minterms, so you can pick the one with the lowest cost.

1. Express "covering all minterms" as a Boolean formula
2. Each minterm needs at least one of its covering PIs
3. Formula in CNF (product of sums)
4. Convert to DNF to see all possible covers
5. Choose cover with minimum cost

Why it works: The CNF formula is satisfiable if and only if we can cover all minterms. Each satisfying assignment is a valid cover.

Petrick's Method: Example

Example: Prime implicants P_1, P_2, P_3, P_4 cover minterms as follows:

	m_1	m_2	m_3	m_4
P_1	$\times$	$\times$		
P_2	$\times$		$\times$	
P_3		$\times$	$\times$	$\times$
P_4			$\times$	$\times$

Coverage formula (CNF):

- m_1: $P_1 + P_2$ (needs P_1 OR P_2)
- m_2: $P_1 + P_3$ (needs P_1 OR P_3)
- m_3: $P_2 + P_3 + P_4$
- m_4: $P_3 + P_4$

Formula: $(P_1 + P_2)(P_1 + P_3)(P_2 + P_3 + P_4)(P_3 + P_4)$

Petrick's Method: Solving

Example (continued): **Expand to DNF:**

$$
\begin{aligned}
&(P_1 + P_2)(P_1 + P_3)(P_2 + P_3 + P_4)(P_3 + P_4) \\
&= (P_1 + P_2P_3)(P_2 + P_3 + P_4)(P_3 + P_4) \\
&= P_1(P_3 + P_4) + P_2P_3(P_3 + P_4) \\
&= P_1P_3 + P_1P_4 + P_2P_3
\end{aligned}
$$

Possible covers:

1. P_1P_3 (2 implicants)
2. P_1P_4 (2 implicants)
3. P_2P_3 (2 implicants)

All have same cost! Choose any: $f = P_1 + P_3$ (for example)

Warning: CNF to DNF expansion can explode exponentially!

Quine-McCluskey: Complete Example

Example: Minimize $f(A, B, C, D) = \sum m(0, 1, 2, 5, 6, 7, 8, 9, 10, 14)$

Group	Minterm	Binary	→	Comb 1	→	Comb 2
0	0	0000		0,1: 000–		0,1,8,9: –00–
1	1	0001		0,2: 00–0		0,2,8,10: –0–0
	2	0010		0,8: –000		
	8	1000		1,9: 100–		
2	5	0101		2,6: 0–10		2,6,10,14: ––10
	6	0110		2,10: –010		
	9	1001		5,7: 01–1		
	10	1010		8,9: 100–		
3	7	0111		8,10: 10–0		
	14	1110		6,7: 011–		
				6,14: –110		
				10,14: 1–10		

Q-M Example: Finding Minimal Cover

Prime implicants:

- P_1: –00– covers $\{0, 1, 8, 9\}$
- P_2: –0–0 covers $\{0, 2, 8, 10\}$
- P_3: ––10 covers $\{2, 6, 10, 14\}$
- P_4: 01–1 covers $\{5, 7\}$

Prime implicant chart:

	0	**1**	**2**	**5**	**6**	**7**	**8**	**9**	**10**	**14**
P_1	×	×					×	×		
P_2	×		×				×		×	
P_3			×		×				×	×
P_4				×		×				

- P_4 is essential (only covers $\{5, 7\}$)
- After selecting P_4, need to cover $\{0, 1, 2, 6, 8, 9, 10, 14\}$

Minimal solution: $f = P_1 + P_3 + P_4$ *or* $f = P_2 + P_3 + P_4$

Complexity of Minimization

Theorem 11: Boolean function minimization is **NP-complete**.

What this means:

- No polynomial-time algorithm known for optimal minimization
- Problem difficulty grows exponentially with function size
- For large functions, we must accept suboptimal solutions
- Heuristics and approximations are necessary

Historical context:

- 1950s-1970s: Minimization critical (gates expensive)
- Quine-McCluskey (1956): First systematic method
- ESPRESSO (1984): Major breakthrough in heuristics
- Modern era: Focus shifted to power/delay over gate count

Modern Minimization Tools

Industrial-strength tools:

ESPRESSO:

- Heuristic multi-level minimizer
- Handles 100+ variables
- Used in major CAD tools

ABC (Berkeley):

- Academic tool for logic synthesis
- Advanced algorithms for large designs

Commercial tools:

- Synopsys Design Compiler
- Cadence Genus

Modern approach:

- Technology mapping (fit to available gates)
- Timing-driven optimization
- Power minimization
- Multi-objective optimization

Practical Minimization Strategy

Recommended approach by function size:

2-3 variables:
- Use K-maps (instant, visual)
- Or simple algebra

4 variables:
- K-maps work well
- Good for learning and verification

5-6 variables:
- K-maps possible but tedious
- Q-M algorithm or tools

7+ variables:
- Use CAD tools (ESPRESSO, ABC)
- Heuristic methods
- Accept near-optimal solutions

Always:
- Verify by expanding result
- Check against original truth table
- Test edge cases

Summary: Minimization Techniques

What we've mastered:

1. **Why minimize:** Cost, power, speed, clarity
2. **Gray code:** Foundation for K-map adjacency
3. **K-maps:** Visual method for 2-5 variables
   - Grouping rules and strategies
   - Wrap-around and corners
   - Don't-care optimization
4. **Algebraic methods:** Laws, consensus, multi-level, factoring
5. **Quine-McCluskey:**
   - Systematic prime implicant generation
   - Prime implicant chart
   - Essential implicants
6. **Petrick's method:** Minimum cover selection
7. **Minimization complexity:** NP-complete
8. **Practical tools:** ESPRESSO, ABC

Next: Zhegalkin polynomials (ANF) — a completely different normal form using XOR!

Algebraic Normal Form

Motivation: A Different Representation

So far, we've built Boolean functions using AND ($\wedge$), OR ($\vee$), and NOT ($\overline{x}$).

This led us to:

- DNF and CNF (canonical forms)
- K-maps and Quine-McCluskey (minimization)
- Circuit design with standard gates

Question: Can we represent Boolean functions using only AND ($\wedge$) and XOR ($\oplus$)?

That is, without OR and NOT operations?

The answer is yes, and this leads to a completely different algebraic structure with important theoretical and practical applications.

Introducing Algebraic Normal Form

Definition 28: An *Algebraic Normal Form* (ANF), also called a *Zhegalkin polynomial*, represents a Boolean function as a polynomial over $\mathbb{F}_2$ (the field $\{0, 1\}$) using:

- XOR ($\oplus$) for addition (modulo 2)
- AND ($\wedge$) for multiplication
- Constants 0 and 1

Example: The function $f(x, y, z) = xy \oplus xz \oplus yz \oplus x \oplus 1$ is in ANF.

Term structure:

- xy, xz, yz are degree-2 terms (quadratic)
- x is a degree-1 term (linear)
- 1 is a degree-0 term (constant)

The algebraic degree of this function is 2.

Notation: We write XOR as $\oplus$ or $+$, and AND as $\wedge$, $\cdot$, or by juxtaposition (xy).

Properties of XOR

Property	OR	XOR
Characteristic	Idempotent $x \vee x = x$	Self-inverse $x \oplus x = 0$
Negation	Requires NOT	$\overline{x} = x \oplus 1$

These differences enable important applications:

Cryptography 🔐
- Stream cipher design
- S-box analysis (DES, AES)
- Algebraic attacks
- Linear cryptanalysis resistance

Coding Theory 📡
- Error-correcting codes
- Reed-Muller codes
- Parity check matrices
- LDPC codes

Note: In $\mathbb{F}_2$, XOR corresponds to addition and AND to multiplication. ANF treats Boolean functions as polynomials over this field.

ANF Structure

Example: The function $f(x, y, z) = xy \oplus xz \oplus yz \oplus x \oplus 1$ is in ANF.

Terms:

- xy — degree 2 (product of 2 variables)
- xz — degree 2
- yz — degree 2
- x — degree 1 (linear term)
- 1 — degree 0 (constant)

The function has algebraic degree 2.

Note: In ANF, we write addition (XOR) as $\oplus$ or simply $+$.

Multiplication (AND) can be written as $\wedge$, $\cdot$, or juxtaposition (*e.g.*, xy).

Comparing ANF and DNF

Property	DNF	ANF
Operations	AND, OR, NOT	AND, XOR only
Structure	OR of AND-terms	XOR of AND-terms
Example	$xy \vee \overline{x}z$	$xy \oplus xz \oplus 1$
Canonical form	Yes (via minterms)	Yes (unique)
Representations	Many variants possible	Exactly one
Negation	Explicit ($\overline{x}$)	$x \oplus 1$
$x + x =$	x (idempotent)	0 (self-inverse)
Identity element	$x \vee 0 = x$	$x \oplus 0 = x$
Degree notion	Not applicable	Max monomial size
Primary use	Circuit synthesis	Cryptographic analysis

Uniqueness of ANF Representation

Theorem 12: Every Boolean function $f : \{0,1\}^n \to \{0,1\}$ has *exactly one* ANF representation.

Proof: Consider Boolean functions as a vector space over $\mathbb{F}_2$:

1. There are 2^{2^n} Boolean functions on n variables
2. There are 2^n possible monomials, giving 2^{2^n} possible ANFs
3. The mapping ANF $\to$ Boolean function is a linear bijection over $\mathbb{F}_2$

Therefore, each function has exactly one ANF representation. $\square$

Significance: Unlike DNF/CNF (which have multiple equivalent forms), ANF is always canonical.

This eliminates the need for minimization in the ANF context.

Note: The uniqueness follows from the linear algebra structure over $\mathbb{F}_2$.
The Möbius transform (used in Pascal's triangle method) provides an explicit bijection.

Monomials in ANF

Example: For $n = 3$ variables (x, y, z):

Degree	Count	Monomials
0	1	1
1	3	x, y, z
2	3	xy, xz, yz
3	1	xyz

Total: $2^3 = 8$ possible monomials (including constant 1).

Each monomial either appears in ANF or doesn't $\longrightarrow 2^8 = 256$ possible ANFs for 3 variables.

Note: For n variables, there are $\sum_{k=0}^{n} \binom{n}{k} = 2^n$ possible monomials.

Algebraic Degree

Definition 29: The *algebraic degree* of a Boolean function in ANF is the maximum number of variables in any monomial with coefficient 1.

Example:

Function	ANF	Degree
NOT	$f(x) = x \oplus 1$	1
XOR	$f(x, y) = x \oplus y$	1
AND	$f(x, y) = xy$	2
NAND	$f(x, y) = xy \oplus 1$	2
Majority	$f(x, y, z) = xy \oplus xz \oplus yz$	2
Full ANF	$f(x, y, z) = xyz \oplus xy \oplus \ldots \oplus 1$	3

Cryptographic Significance of Algebraic Degree

Low algebraic degree indicates vulnerability to attacks.

Functions with degree $d < \frac{n}{2}$ are susceptible to:

- Linearization attacks
- Cube attacks
- Higher-order differential cryptanalysis

Example: The AES S-box is a nonlinear bijection $\mathbb{F}_2^8 \to \mathbb{F}_2^8$ with algebraic degree 7 (approaching the maximum of 8 for 8-bit functions). This high degree provides resistance to algebraic attacks.

Methods for Computing ANF

We will examine three methods for computing ANF from truth tables:

Method	Complexity	Variables	Difficulty	Best suited for
Direct computation	$O(2^{2n})$	1-3	High	Theoretical understanding
Pascal's triangle	$O(2^n)$	1-5	Medium	Hand calculations
Karnaugh map	$O(2^n)$	2-4	Low	Visual intuition

Learning approach:

1. Begin with K-map method (most intuitive)
2. Master Pascal's triangle (most practical)
3. Understand direct method (theoretical foundation)

Each method provides different insights into the structure of ANF.

Method 1: Direct Computation

Idea: Solve a system of linear equations over $\mathbb{F}_2$ (arithmetic mod 2).

For function $f : \mathbb{F}_2^n \to \mathbb{F}_2$, we want to find coefficients a_S such that:

$$f(x_1, \ldots, x_n) = \bigoplus_{S \subseteq \{1,\ldots,n\}} \left(a_S \prod_{i \in S} x_i \right)$$

where $\oplus$ denotes XOR and $\prod$ denotes AND.

Each row of the truth table gives one linear equation!

Remember: In $\mathbb{F}_2$, addition is XOR:

- $1 \oplus 1 = 0 \oplus 0 = 0$
- $1 \oplus 0 = 0 \oplus 1 = 1$

Direct Computation: Example

Example: Find ANF for $f(x, y)$ with truth table:

x	y	$f(x, y)$
0	0	1
0	1	1
1	0	1
1	1	0

(Solution on next slide)

Direct Computation: Example [2]

Let ANF be: $f(x, y) = a_0 \oplus a_1 x \oplus a_2 y \oplus a_3 xy$

From truth table, substitute each (x, y) to get 4 equations over $\mathbb{F}_2$:

(x, y)	**Substituted equation**		f
(0,0)	$a_0 \oplus (a_1 \cdot 0) \oplus (a_2 \cdot 0) \oplus (a_3 \cdot 0 \cdot 0) = a_0$	$=$	1
(0,1)	$a_0 \oplus (a_1 \cdot 0) \oplus (a_2 \cdot 1) \oplus (a_3 \cdot 0 \cdot 1) = a_0 \oplus a_2$	$=$	1
(1,0)	$a_0 \oplus (a_1 \cdot 1) \oplus (a_2 \cdot 0) \oplus (a_3 \cdot 1 \cdot 0) = a_0 \oplus a_1$	$=$	1
(1,1)	$a_0 \oplus a_1 \oplus a_2 \oplus a_3$	$=$	0

Solving step-by-step:

- Equation 1: $a_0 = 1$
- Equation 2: $(a_0 \oplus a_2 = 1) \Rightarrow (1 \oplus a_2 = 1) \Rightarrow a_2 = 0$
- Equation 3: $(a_0 \oplus a_1 = 1) \Rightarrow (1 \oplus a_1 = 1) \Rightarrow a_1 = 0$
- Equation 4: $(1 \oplus 0 \oplus 0 \oplus a_3 = 0) \Rightarrow (1 \oplus a_3 = 0) \Rightarrow a_3 = 1$

Result: $f(x, y) = 1 \oplus xy$, which is equivalent to $f = x \overline{\wedge} y$.

Method 2: Pascal's Triangle Method

Fast systematic method using a butterfly/pyramid pattern!

Algorithm:

1. Write function values f in a column (in binary order: 000, 001, 010, ...)
2. Create next column: XOR each adjacent pair $(f_i \oplus f_{i+1})$
3. Repeat step 2 until only one value remains
4. The **first value** of each column is an ANF coefficient

Note: Also called the *Butterfly method* or *ANF transform*.

Pattern: Coefficients appear in binary order: 1, z, y, yz, x, xz, xy, xyz

Pascal's Triangle Method: Example

Find ANF for $f(x, y, z) = \sum m(0, 1, 3, 7)$:

xyz	f	a_1	a_z	a_y	a_{yz}	a_x	a_{xz}	a_{xy}	a_{xyz}
000	1	$\boxed{1}$	0	$\boxed{1}$	$\boxed{1}$	$\boxed{1}$	0	$\boxed{1}$	0
001	1	1	1	0	0	1	1	1	
010	0	0	1	0	1	0	0		
011	1	1	1	1	1	0			
100	0	0	0	0	1				
101	0	0	0	1					
110	0	0	1						
111	1	1							

ANF: $f = 1 \oplus y \oplus yz \oplus x \oplus xy$

Pascal's Triangle: Why It Works

The transformation computes the *Möbius transform* over the Boolean lattice:

$$a_S = \bigoplus_{T \subseteq S} f(T)$$

Each coefficient a_S is the XOR of all function values $f(T)$ where $T \subseteq S$.

Example: For a_{xy}, corresponding to input 110 = $\{x, y\}$:

$$\begin{aligned} a_{xy} &= f(000) \oplus f(010) \oplus f(100) \oplus f(110) \\ &= 1 \oplus 0 \oplus 0 \oplus 0 = 1 \end{aligned}$$

Historical note: Related to Reed-Muller codes and Fast Walsh-Hadamard Transform!

Method 3: Karnaugh Map Method

Use K-map regions to identify monomials in ANF.

Algorithm:

1. Fill K-map with function values
2. For each possible monomial:
   - Identify its rectangular region in the K-map
   - If the top-left cell equals 1, include the monomial
3. Combine all selected monomials using XOR

Key differences from minimization:

- Check *all* rectangular regions, not just maximal ones
- Combine terms with XOR instead of OR
- Top-left cell determines inclusion

K-Map Method: Example

Example: Find ANF for $f(x, y) = \sum m(1, 2)$:

K-map:

$x \backslash y$	0	1
0	0 (0)	1 (1)
1	1 (2)	0 (3)

Monomial analysis:

1. Constant (all cells): top-left = 0 $\Rightarrow$ exclude
2. x (row $x = 1$): top-left = 1 $\Rightarrow$ include
3. y (column $y = 1$): top-left = 1 $\Rightarrow$ include
4. xy (cell at 11): value = 0 $\Rightarrow$ exclude

Result ANF: $f = x \oplus y$

K-Map Method: Another Example

Example: Find ANF for $f(x, y, z) = \sum m(0, 1, 3, 7)$

TODO

Groups to check:

1. **Constant** (all 8 cells): $1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 0 \Rightarrow$ No constant term
2. $\boldsymbol{z}$ (columns 01, 11): Top-left at (0, 01) is 1 $\Rightarrow$ Include z
3. $\boldsymbol{y}$ (columns 10, 11): Top-left at (0, 10) is 0 $\Rightarrow$ No
4. $\boldsymbol{x}$ (bottom row): Top-left at (1, 00) is 0 $\Rightarrow$ No
5. $\boldsymbol{yz}$ (column 11): Top-left at (0, 11) is 1 $\Rightarrow$ Include yz
6. $\boldsymbol{xyz}$ (cell at $x = 1$, $yz = 11$): Value is 1 $\Rightarrow$ Include xyz

Result: $f = 1 \oplus z \oplus yz \oplus xyz$

Comparison of ANF Construction Methods

Method	Pros	Cons	Best for
📐 **Direct computation**	• Systematic • *Always works* • Clear math	• Tedious • Many equations • Error-prone	1-2 variables, learning
🔺 **Pascal's triangle**	• *Fast* • Mechanical • Scales well	• Less intuitive • Needs practice • Easy to miscount	3-5 variables, computation
🗺️ **K-map method**	• Visual • *Intuitive* • Good for patterns	• Only 2-4 variables • Can miss groups • Needs care	2-4 variables, understanding

Recommendation: Learn all three! Use Pascal's triangle for exams, K-map for intuition.

Summary of ANF Properties

Key properties of ANF:

1. **Uniqueness:** Every Boolean function has exactly one ANF
2. **Completeness:** ANF can represent any Boolean function
3. **Operations:** Uses only XOR and AND (no NOT needed)
4. **Algebraic degree:** Maximum degree of monomials
5. **Self-inverse:** $x \oplus x = 0$ (unlike $x \vee x = x$)
6. **Linearity:** XOR is linear over $\mathbb{F}_2$

ANF in Cryptography

Why cryptographers care about ANF?

Algebraic Attacks

Express cipher as polynomial system over $\mathbb{F}_2$:

- Each round forms polynomial equations
- Solve using Gröbner basis algorithms
- Low degree enables efficient attacks

Note: If an S-box has degree 2, attackers can linearize the system and solve in polynomial time.

S-Box Design Criteria

Secure S-boxes require:

- High algebraic degree (nonlinearity)
- Balanced output distribution
- Absence of linear structures

Note: The AES S-box has degree 7 (near maximum of 8), providing resistance to algebraic attacks.

ANF in Cryptography [2]

Stream Cipher Design

Boolean functions in stream ciphers:

- Filter functions for LFSRs
- Combining functions
- Must resist correlation and algebraic attacks

Degree requirements depend on cipher structure.

Important Function Classes

- **Bent functions:** Maximum nonlinearity
- **Balanced functions:** Equal 0s and 1s in output
- **Correlation-immune:** Statistical attack resistance
- **Resilient:** Balanced and correlation-immune

Case study: The E0 stream cipher (used in Bluetooth) was cryptanalyzed due to insufficient algebraic degree in its combining function.

Converting Between ANF and DNF

Two approaches for converting between representations:

Method	Advantages	Disadvantages
Algebraic Direct application of identities	• Direct transformation • Shows structural relationships	• Complex algebraic expansions • Error-prone for large functions • Exponential term growth
Via truth table Truth table as intermediate form	• Systematic procedure • Always succeeds • Clear verification steps	• Requires complete enumeration • Additional computational step

Recommended approach: Use truth table as intermediate representation.

This method is systematic, reliable, and suitable for examination contexts.

Note: Direct algebraic conversion requires expanding $x \oplus y = (x \wedge \overline{y}) \vee (\overline{x} \wedge y)$, which becomes complex for multi-variable functions.

Converting ANF to DNF

Example: Convert $f = x \oplus y \oplus xy$ (ANF) to DNF

Step 1: Build truth table by evaluating ANF:

x	y	$f = x \oplus y \oplus xy$	**Minterm**
0	0	$0 \oplus 0 \oplus 0 = 0$	—
0	1	$0 \oplus 1 \oplus 0 = 1$	$\overline{x}y$
1	0	$1 \oplus 0 \oplus 0 = 1$	$x\overline{y}$
1	1	$1 \oplus 1 \oplus 1 = 1$	xy

Step 2: Read minterms where $f = 1$:

DNF: $f = \overline{x}y \vee x\overline{y} \vee xy = \sum m(1, 2, 3)$

Converting DNF to ANF

Example: Convert $f = \overline{x}y \vee x\overline{y}$ (DNF) to ANF

Step 1: Build truth table by evaluating DNF

Step 2: Apply Pascal's triangle method to get ANF:

xy	f	a_1	a_x	a_y	a_{xy}
00	0	0	$\boxed{1}$	$\boxed{1}$	0
01	1	1	0	1	
10	1	1	1		
11	0	0			

ANF: $f = x \oplus y$ (the XOR function!)

Summary: Algebraic Normal Form

Key concepts covered:

Theoretical foundations

1. ANF as polynomials over $\mathbb{F}_2$
2. Representation using XOR and AND
3. Unique canonical form
4. Algebraic degree as security parameter

Computational methods

1. Pascal's triangle (efficient)
2. Karnaugh map (intuitive)
3. Direct computation (theoretical)
4. Conversion via truth tables

Applications

- Cryptographic analysis: S-box evaluation, algebraic attacks
- Coding theory: Reed-Muller codes, error correction
- Hardware optimization: XOR-based circuit design

Looking ahead: Which sets of operations are sufficient to express all Boolean functions?

Functional Completeness

The Fundamental Question

We've seen many Boolean operations: AND, OR, NOT, XOR, NAND, NOR, implication, equivalence...

Central question: Which sets of operations can express *every* Boolean function?

For example:

- Can we build everything using only $\{\wedge, \vee, \neg\}$? ✓
- Can we build everything using only $\{\text{NAND}\}$? ✓
- Can we build everything using only $\{\wedge, \vee\}$? ✗

This leads to the concept of *functional completeness* — a cornerstone of Boolean algebra and digital circuit design.

Functional Closure

Definition 30: Let F be a set of Boolean functions. The *functional closure* $[F]$ is the smallest set containing F and closed under:

1. **Composition:** If $f, g \in [F]$, then $h(x) = f(g(x)) \in [F]$
2. **Identification:** If $f(x_1, \ldots, x_n) \in [F]$, then $f(x, x, x_3, \ldots, x_n) \in [F]$
3. **Permutation:** If $f(x_1, \ldots, x_n) \in [F]$, then $f\left(x_{\sigma(1)}, \ldots, x_{\sigma(n)}\right) \in [F]$ for any permutation σ
4. **Introduction of fictitious variables:** If $f(x_1, \ldots, x_n) \in [F]$, then $f(x_1, \ldots, x_n, x_{n+1}) \in [F]$

Informally, $[F]$ contains all functions you can build by *combining* functions from F.

Note: The closure operations keep the important structure while allowing you combine functions freely.

Functional Completeness

Definition 31: A set F of Boolean functions is *functionally complete* (or *universal*) if its closure $[F]$ contains all possible Boolean functions.

Equivalently: every Boolean function can be expressed using only operations from F.

Example: **Complete sets:**

- $\{\wedge, \vee, \neg\}$ — the classical basis
- $\{\wedge, \neg\}$ — AND with negation
- $\{\vee, \neg\}$ — OR with negation
- $\{\text{NAND}\}$ — NAND alone (Sheffer stroke)
- $\{\text{NOR}\}$ — NOR alone (Peirce arrow)

Incomplete sets:

- $\{\wedge, \vee\}$ — cannot produce NOT
- $\{\oplus\}$ — only produces linear functions
- $\{\neg\}$ — only produces constants and negations

Why Functional Completeness Matters

Circuit Design

Functionally complete sets determine which gates are sufficient to build any digital circuit.

Note: NAND and NOR gates are universal — entire processors can be built using only NAND gates!

Programming Languages

Programming languages must provide functionally complete sets of operators.

Example: C provides: `&&`, `||`, `!`, `^`
Each subset {`&&`, `!`} or {`||`, `!`} is complete.

Practical constraint: Manufacturing circuits with a single gate type (NAND or NOR) simplifies design, testing, and production.

Post's Five Classes

Emil Post (1941) identified five classes of Boolean functions that restrict expressiveness:

Definition 32: A Boolean function $f : \{0,1\}^n \to \{0,1\}$ is:

1. $\boldsymbol{T_0}$ **(preserves 0):** $f(0,0,\ldots,0) = 0$
2. $\boldsymbol{T_1}$ **(preserves 1):** $f(1,1,\ldots,1) = 1$
3. $\boldsymbol{S}$ **(self-dual):** $f(\overline{x}_1,\ldots,\overline{x}_n) = \overline{f(x_1,\ldots,x_n)}$
4. $\boldsymbol{M}$ **(monotone):** $x \leq y \Rightarrow f(x) \leq f(y)$ (bitwise comparison)
5. $\boldsymbol{L}$ **(linear):** f has ANF of degree ≤ 1 (no AND terms)

Note: These classes form a complete classification: if F is fully contained ($\subseteq$) in any class, then $[F]$ cannot contain *all* Boolean functions.

Post's Classes: Examples

Function	T_0	T_1	S	M	L
Constant 0	✓	✗	✗	✓	✓
Constant 1	✗	✓	✗	✓	✓
Identity x	✓	✓	✓	✓	✓
NOT $\overline{x}$	✗	✗	✓	✗	✓
AND $x \wedge y$	✓	✓	✗	✓	✗
OR $x \vee y$	✓	✓	✗	✓	✗
XOR $x \oplus y$	✓	✗	✗	✗	✓
NAND $\overline{x \wedge y}$	✗	✗	✗	✗	✗
NOR $\overline{x \vee y}$	✗	✗	✗	✗	✗
Implication $x \to y$	✗	✓	✗	✗	✗

Observation: NAND and NOR belong to *none* of the five Post classes!

Understanding Post's Classes

T_0 and T_1 (Preserving Constants)

Functions that map all-0s to 0 (T_0) or all-1s to 1 (T_1).

- AND, OR are in both T_0 and T_1
- XOR is in T_0 but not T_1
- NOT is in neither

Note: If all functions preserve 0, closure cannot produce constants.

S (Self-Dual Functions)

Swapping inputs and output gives the same function.

$f(\overline{x}) = \overline{f(x)}$

- NOT is self-dual: $\overline{\overline{x}} = x$
- Median: $\text{med}(x, y, z) = xy + yz + xz$

Note: Self-dual functions satisfy $f = \overline{f} \circ \neg$

Understanding Post's Classes [2]

M (Monotone Functions)

Increasing inputs never decrease output.

If $x_i \leq y_i$ for all i, then $f(x) \leq f(y)$.

- AND, OR are monotone
- NOT, XOR are not monotone

Example: For function OR:

- $(0,0) < (0,1)$ and $\mathrm{OR}(0,0) < \mathrm{OR}(0,1)$ ✓

L (Linear Functions)

ANF has degree ≤ 1 (only XOR and constants).

$f = a_0 \oplus a_1 x_1 \oplus \ldots \oplus a_n x_n$

- Constants, NOT, XOR are linear
- AND, OR are not linear (degree 2)

Note: Linear functions form a vector space over $\mathbb{F}_2$.

Key insight: Each Post class is *closed under composition* — combining functions from a class stays within that class.

Post's Criterion

Theorem 13: A set F of Boolean functions is functionally complete if and only if F contains at least one function that does *not* belong to each of the five Post classes:

$$F \text{ is complete} \iff \begin{cases} \exists f \in F : f \notin T_0 \\ \exists f \in F : f \notin T_1 \\ \exists f \in F : f \notin S \\ \exists f \in F : f \notin M \\ \exists f \in F : f \notin L \end{cases}$$

Note: Each f can be different, or the *same* function can escape multiple classes.

Equivalently: F is complete $\iff$ F is not contained in any Post class.

Practical test: To prove F is complete, find five functions (possibly the same) escaping each class.

Post's Criterion: Intuition

Why does escaping all five classes guarantee completeness?

Idea: Each Post class represents a "weakness" — *inability to express* certain functions:

- $\boldsymbol{T_0}$**:** Cannot create constant 1 from all-0 input
- $\boldsymbol{T_1}$**:** Cannot create constant 0 from all-1 input
- $\boldsymbol{S}$**:** Cannot break symmetry between x and $\overline{x}$
- $\boldsymbol{M}$**:** Cannot decrease output when input increases
- $\boldsymbol{L}$**:** Cannot create nonlinear interactions (AND-like)

If F escapes all restrictions, it has enough "power" to build any function.

Note: The proof constructs NOT, AND, and OR from functions escaping the five classes, establishing completeness via a known complete set — Standard Boolean basis $\{\wedge, \vee, \neg\}$.

Post's Criterion: Proof Sketch

The full proof is technical, but the key steps are:

Step 1: Show that if $F \subseteq T_0$, then $[F] \subseteq T_0$ (similarly for T_1, S, M, L).

Proof: Composition preserves each class. For example, if $f, g \in T_0$, then:

$$h(0, \ldots, 0) = f(g(0, \ldots, 0), \ldots) = f(0, \ldots, 0) = 0$$

Step 2: If F escapes all five classes, construct $\{\wedge, \neg\}$ from F.

This involves:

- Using non-T_0 and non-T_1 to create constants
- Using non-S to break self-duality
- Using non-M to enable decreasing behavior
- Using non-L to create nonlinear terms

Step 3: Since $\{\wedge, \neg\}$ is complete, $[F]$ contains all functions. ■

Verifying Completeness: Examples

Example: **Is $\{\wedge, \vee, \neg\}$ complete?**

Check each class:

- NOT $\notin T_0$: NOT(0) = 1 $\neq$ 0 ✓
- NOT $\notin T_1$: NOT(1) = 0 $\neq$ 1 ✓
- AND $\notin S$: non-self-dual ✓
- NOT $\notin M$: NOT is not monotone ✓
- AND $\notin L$: AND has degree 2 ✓

Conclusion: Complete! ✓

Verifying Completeness: Examples [2]

Example: **Is $\{\mathbf{NAND}\}$ complete?**

NAND(x, y) = $\overline{x \wedge y}$:

- NAND(0,0) = 1 ≠ 0 $\Rightarrow \notin T_0$ ✓
- NAND(1,1) = 0 ≠ 1 $\Rightarrow \notin T_1$ ✓
- NAND($\overline{x}, \overline{y}$) ≠ $\overline{\mathrm{NAND}(x, y)} \Rightarrow \notin S$ ✓
- NAND(0,1) = 1 > NAND(1,1) = 0 $\Rightarrow \notin M$ ✓
- ANF has degree 2 $\Rightarrow \notin L$ ✓

Conclusion: Complete! ✓

Incomplete Sets: Examples

Example: **Is $\{\wedge, \vee\}$ complete?**

- AND $\in T_0$: $\mathrm{AND}(0,0) = 0$
- OR $\in T_0$: $\mathrm{OR}(0,0) = 0$
- All functions in closure stay in T_0
- Cannot escape T_0 (need function with $f(0, \ldots, 0) = 1$)

Conclusion: Incomplete. Stuck in T_0. ✗

Example: **Is $\{\oplus, \neg\}$ complete?**

- XOR $\in L$: degree 1
- NOT $\in L$: $\neg x = x \oplus 1$, degree 1
- All functions in closure stay in L
- Cannot escape L (need nonlinear function like AND)

Conclusion: Incomplete. Stuck in L. ✗

Complete Sets in Practice

Common functionally complete sets:

Set	Applications	Notes
$\{\wedge, \vee, \neg\}$	Textbooks, theory	Classical basis
$\{\wedge, \neg\}$	TTL logic families	Two-level logic
$\{\vee, \neg\}$	Alternative basis	Dual to {AND, NOT}
$\{\neg, \Rightarrow\}$	Logic programming	Implication-based
$\{\text{NAND}\}$	Digital circuits	Single gate type
$\{\text{NOR}\}$	Digital circuits	Single gate type
$\{\oplus, \wedge, 1\}$	ANF synthesis	Algebraic forms

Design principle: Minimal complete sets reduce hardware complexity and manufacturing costs.

Definability and Applications

Definition 33: A function f is *definable* from set F if $f \in [F]$.

- XOR is definable from {AND, OR, NOT}:
  $x \oplus y = (x \wedge \overline{y}) \vee (\overline{x} \wedge y)$
- NOT is definable from {NAND}:
  $\overline{x} = \mathrm{NAND}(x, x)$
- AND is definable from {NAND}:
  $x \wedge y = \overline{\mathrm{NAND}(x, y)} = \mathrm{NAND}(\mathrm{NAND}(x, y), \mathrm{NAND}(x, y))$

Applications:

1. **Circuit synthesis:** Convert expressions to available gate types
2. **Gate minimization:** Replace complex gates with universal gates
3. **Hardware verification:** Ensure gate library is sufficient
4. **Compiler optimization:** Transform logical expressions efficiently

Building Functions from NAND

Since $\{\text{NAND}\} = \{\overline{\wedge}\}$ is complete, we can build any function using only NAND gates.

Example: **Construct basic gates from NAND:**

Gate	Formula	NAND equivalent
NOT	$\neg x$	$x \,\overline{\wedge}\, x$
AND	$x \wedge y = \neg(x \,\overline{\wedge}\, y)$	$(x \,\overline{\wedge}\, y) \,\overline{\wedge}\, (x \,\overline{\wedge}\, y)$
OR	$x \vee y = \neg(\neg x \wedge \neg y)$	$(x \,\overline{\wedge}\, x) \,\overline{\wedge}\, (y \,\overline{\wedge}\, y)$
XOR	$x \oplus y = (x \wedge \neg y) \vee (\neg x \wedge \neg y)$	$(x \,\overline{\wedge}\, (x \,\overline{\wedge}\, y)) \,\overline{\wedge}\, (y \,\overline{\wedge}\, (x \,\overline{\wedge}\, y))$

Trade-off: Using only NAND gates may increase circuit depth and gate count compared to mixed-gate designs.

Note: Similar constructions exist for NOR gates, providing alternative universal implementations.

Summary: Functional Completeness

Key concepts:

1. **Functional closure** $[F]$: all functions expressible using F
2. **Functional completeness**: $[F]$ contains all Boolean functions
3. **Post's five classes**: T_0, T_1, S, M, L (each closed under composition)
4. **Post's criterion**: F is complete $\Leftrightarrow$ F escapes all five classes
5. **Universal gates**: NAND and NOR are individually complete

Practical applications:

- Circuit design with minimal gate types
- Optimization of Boolean expressions
- Hardware verification and gate library design
- Understanding expressiveness limits

Looking ahead: How do we implement these Boolean functions in actual hardware?

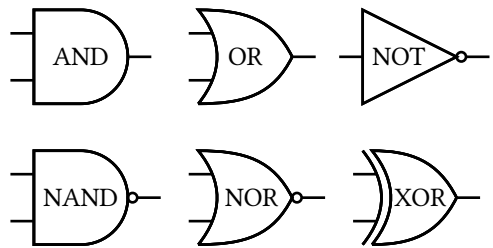
Next: Digital circuits, gates, and flip-flops.

Digital Circuits

Digital Logic Circuits

Definition 34: A *logic gate* is a physical device that implements a Boolean function, taking binary inputs and producing a binary output.

Gate	Formula	Description
AND	$A \wedge B$	Outputs 1 only when both inputs are 1
OR	$A \vee B$	Outputs 1 when at least one input is 1
NOT	$\neg A$	Outputs the opposite of the input
NAND	$A \overline{\wedge} B$	Outputs 0 only when both inputs are 1
NOR	$A \overline{\vee} B$	Outputs 0 when at least one input is 1
XOR	$A \oplus B$	Outputs 1 when inputs differ
XNOR	$A \equiv B$	Outputs 1 when inputs are the same



Note: NAND and NOR gates are *universal* — any Boolean function can be implemented using only NAND gates (or only NOR gates). For example, to implement AND using NAND:

$$A \wedge B = \neg\neg(A \wedge B) = \neg(A \overline{\wedge} B) = (A \overline{\wedge} B) \overline{\wedge} (A \overline{\wedge} B)$$

Combinational Logic

Definition 35: A *combinational circuit* is a circuit where the output depends only on the current input values, without any memory or state.

Example (Half Adder): Adds two single bits:

- Sum: $S = A \oplus B$
- Carry: $C = A \wedge B$

Example (Full Adder): Adds two bits plus a carry-in:

- Sum: $S = A \oplus B \oplus C_{\text{in}}$
- Carry-out: $C_{\text{out}} = (A \wedge B) \vee (C_{\text{in}} \wedge (A \oplus B))$

Sequential Logic and Memory

Definition 36: A *sequential circuit* is a circuit where the output depends on both current inputs and previous state (memory).

Example (Flip-Flops):

- **SR Latch**: Set-Reset memory element.
- **D Flip-Flop**: Data storage triggered by clock edge.
- **JK Flip-Flop**: Eliminates forbidden state of SR latch.
- **T Flip-Flop**: Toggle flip-flop for counters.